UNIT III
EXCEPTION AND FILE I/O STREAMS

Exceptions – benefits of exception – Types of Exceptions – Errors – Control flow – JVM reaction to Exception – usage of try, catch, throw, final and finally keyword – rethrowing exceptions, exception specification, built in Exceptions – File I/O: Standard Streams – Reading and writing Streams – Byte Array Stream – Data Stream – File Stream- Input and output Stream.

3.1 Exception

An exception is a problem that arises during the execution of a program. When an Exception occurs the normal flow of the program is disrupted and the program/Application terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.

An exception can occur for many different reasons. Following are some scenarios where an exception occurs.

- A user has entered an invalid data.
- A file that needs to be opened cannot be found.
- A network connection has been lost in the middle of communications or the JVM has run out of memory.

3.1.2 Benefits of Exceptions

- **Graceful Termination:** Prevents abrupt program termination, allowing for clean shutdown or recovery.
- **Separation of Concerns:** Separates error-handling code from regular program logic, improving readability and maintainability.
- **Robustness:** Makes programs more resilient to unexpected situations.
- **User-Friendly Feedback:** Allows for presenting meaningful error messages to users instead of cryptic stack traces.

3.1.3 Types of Exceptions

There are three categories of Exceptions.

1. **Checked exceptions** − A checked exception is an exception that is checked (notified) by the compiler at compilation-time, these are also called as compile time exceptions.

```
import java.io.File;
import java.io.FileReader;
public class FilenotFound_Demo {
   public static void main(String args[]) {
      File file = new File("E://file.txt");
      FileReader fr = new FileReader(file);
   }
}
```

If you try to compile the above program, you will get the following exceptions.

Output

```
C:\>javac FilenotFound_Demo.java
FilenotFound_Demo.java:8:      error:      unreported      exception
FileNotFoundException; must be caught or declared to be thrown
      FileReader fr = new FileReader(file);
                      ^
1 error
```

2. **Unchecked exceptions** − An unchecked exception is an exception that occurs at the time of execution.

```
public class Unchecked_Demo {
      public static void main(String args[]) {
      int num[] = {1, 2, 3, 4};
      System.out.println(num[5]);
   }
}
```

If you compile and execute the above program, you will get the following exception.

Output

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
        at Exceptions.Unchecked_Demo.main(Unchecked_Demo.java:8)
```

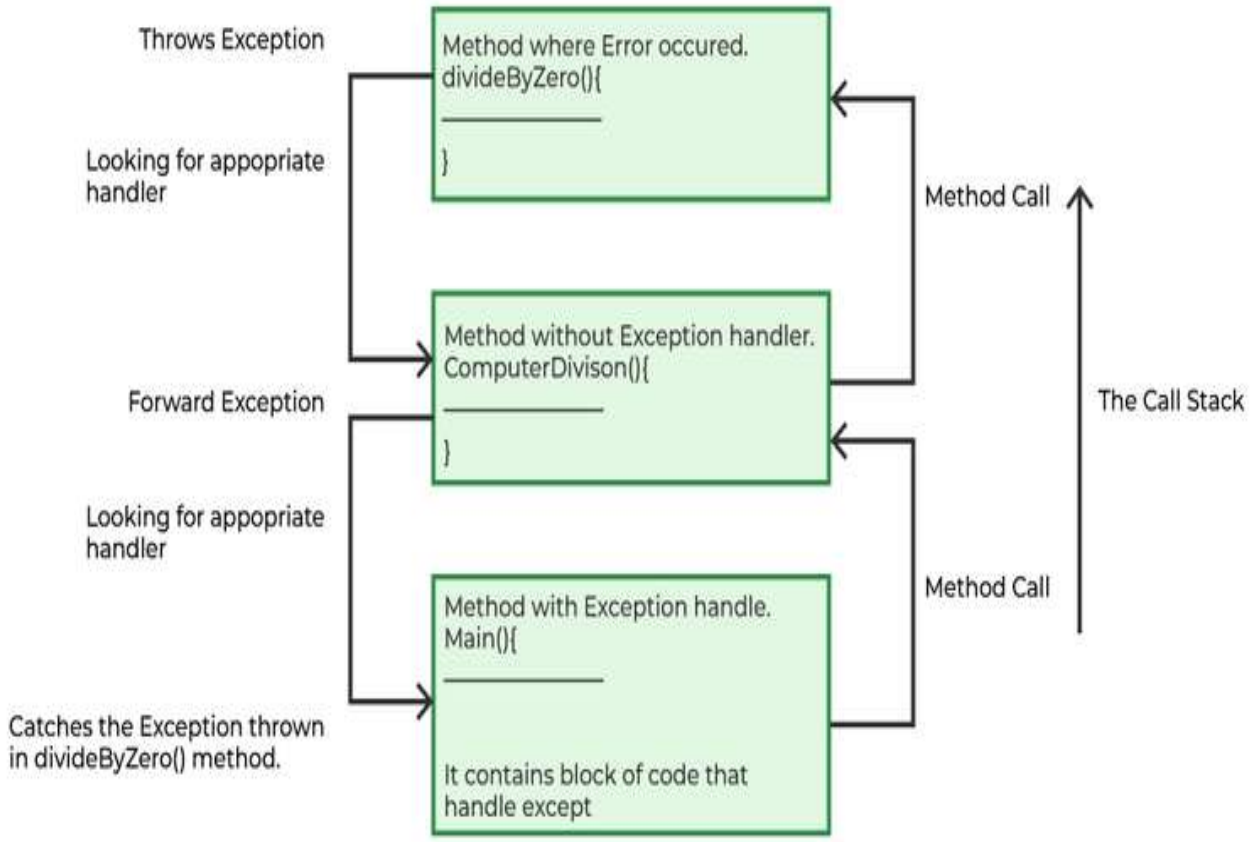
3.1.4 Errors

These are not exceptions at all, but problems that arise beyond the control of the user or the programmer. Errors are typically ignored in your code because you can rarely do anything about an error. For example, if a stack overflow occurs, an error will arise. They are also ignored at the time of compilation.

3.1.5 Control Flow in Exceptions

When an exception occurs:

1. The normal flow of the program is interrupted.
2. Control transfers to the nearest matching catch block.
3. If not caught, it propagates up the call stack.
4. If unhandled, JVM terminates the program.



3.1.6 JVM reaction to Exception

When an Exception occurs, the JVM creates an exception object containing the error name, description, and program state. Creating the exception object and handling it in the run-time system is called throwing an exception. There might be a list of the methods that had been called to get to the method where an exception occurred. This ordered list of methods is called call stack. Now the following procedure will happen:

- The run-time system searches the call stack for an exception handler
- It starts searching from the method where the exception occurred and proceeds backward through the call stack.
- If a handler is found, the exception is passed to it.
- If no handler is found, the default exception handler terminates the program and prints the stack trace.

3.1.7 Usage of try, catch, throw, throws and finally keyword

There are 5 keywords which are used in handling exceptions in Java.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

The code which is likely to cause exceptions is placed in the try block. When an exception occurs, that exception occurred is handled by catch block associated with it. Every try block should be immediately followed either by a catch block or finally block.

Syntax

```
try {
  // Protected code
} catch (ExceptionName e1) {
  // Catch block
}
```

Example

```
// File Name : ExcepTest.java
import java.io.*;
public class ExcepTest {
  public static void main(String args[]) {
    try {
      int a[] = new int[2];
      System.out.println("Access element three :" + a[3]);
    }
  catch (ArrayIndexOutOfBoundsException e) {
      System.out.println("Exception thrown  :" + e);
    }
    System.out.println("Out of the block");
  }
}
```

Output

```
Exception thrown  :java.lang.ArrayIndexOutOfBoundsException: 3
Out of the block
```

3.1.8 Rethrowing Exceptions

You can catch an exception and throw it again:

```
try {
   // risky code
} catch (IOException e) {
```

```
            // log or wrap
            throw e; // rethrows the exception
        }
```

Example:

```
public class RethrowDivideExample {
    // Method that may throw an exception
    static int divide(int a, int b) {
        try {
            return a / b; // May throw ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Caught in divide(): " + e.getMessage());
            throw e; // Rethrowing the same exception
        }
    }
    public static void main(String[] args) {
        try {
            int result = divide(10, 0); // Will cause divide by zero
            System.out.println("Result: " + result);
        }
catch (ArithmeticException e) {
            System.out.println("Caught in main(): " + e.getMessage());
        }
    }
}
```

Output:

```
Caught in divide(): / by zero
Caught in main(): / by zero
```

- divide() tries to divide a by b.
- If b is 0, it throws an ArithmeticException.
- This exception is **caught inside** divide() and then **rethrown**.
- The exception is **caught again** in the main() method.

3.1.9 Common Scenarios of Java Exceptions

1) ArithmeticException

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

2) NullPointerException

If we have a null value in any variable, performing any operation on the variable throws a NullPointerException.

```
String s=null;
System.out.println(s.length());//NullPointerException
```

3) NumberFormatException

The wrong formatting of any value may occur NumberFormatException. Suppose I have a string variable that has characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";
int i=Integer.parseInt(s);//NumberFormatException
```

4) ArrayIndexOutOfBoundsException

If you are inserting any value in the wrong index, it would result in ArrayIndexOutOfBoundsException as shown below:

```
int a[]=new int[5];
a[10]=50; //ArrayIndexOutOfBoundsException
```

3.1.10 Multi-catch block

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

```
public class MultipleCatchBlock1 {
   public static void main(String[] args) {
        try{
            int a[]=new int[5];
            a[5]=30/0;
           }
           catch(ArithmeticException e)                {
              System.out.println("Arithmetic Exception occurs");
             }
           catch(ArrayIndexOutOfBoundsException e)                {
              System.out.println("ArrayIndexOutOfBounds Exception occurs");
             }
           catch(Exception e)                {
              System.out.println("Parent Exception occurs");
             }
           System.out.println("rest of the code");
   }
}
```

Output:

```
Arithmetic Exception occurs
rest of the code
```

3.1.11 Nested try block

The try block within a try block is known as nested try block in java. Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.

Syntax:

```
....
try
{
   statement 1;
   statement 2;
   try
   {
      statement 1;
      statement 2;
   }
   catch(Exception e)
   {
   }
}
catch(Exception e)
{
}
....
```

Example

```
class Nested_Excep{
```

```
 public static void main(String args[]){
  try{
    try{
     System.out.println("going to divide");
     int b =39/0;
    }catch(ArithmeticException e){System.out.println(e);}
       try{
    int a[]=new int[5];
    a[5]=4;
    }catch(ArrayIndexOutOfBoundsException e){System.out.println(e);}
        System.out.println("other statement);
  }catch(Exception e){System.out.println("handeled");}
    System.out.println("normal flow..");
 }
}
```

Output

```
going to divide
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: 5
other statement
normal flow..
```

3.1.12 finally block

Java finally block is a block that is used to execute important code such as closing connection, stream etc. Java finally block is always executed whether exception is handled or not. Java finally block follows try or catch block.

```
public class TestFinallyBlock2{
  public static void main(String args[]){
  try{
   int data=25/0;
   System.out.println(data);
  }
  catch(ArithmeticException e){System.out.println(e);}
  finally{System.out.println("finally block is always executed");}
  System.out.println("rest of the code...");
  }
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
finally block is always executed
rest of the code..
```

3.1.13 throws keyword

When a method wants to throw an exception, the keyword throws is used

Syntax

```
return_type method_name() throws exception_class_name{
//method code
}
```

Program

```
import java.io.*;
class ThrowExample {
  void myMethod(int num)throws IOException, ClassNotFoundException{
```

```
        if(num==1)
          throw new IOException("IOException Occurred");
        else
          throw new ClassNotFoundException("ClassNotFoundException");
      }
    }
    public class Example1{
      public static void main(String args[]){
       try{
        ThrowExample obj=new ThrowExample();
        obj.myMethod(1);
       }catch(Exception ex){
        System.out.println(ex);
       }  } }
```

Output:

java.io.IOException: IOException Occurred

3.1.14 throw keyword

The Java throw keyword is used to explicitly throw an exception.

Syntax

throw new exception(“Exception Ststement”);

Program

```
public class TestThrow1{
  static void validate(int age){
    if(age<18)
     throw new ArithmeticException("not valid");
    else
     System.out.println("welcome to vote");
  }
  public static void main(String args[]){
    validate(13);
    System.out.println("rest of the code...");
 }
}
```

Output:

Exception in thread main java.lang.ArithmeticException:not valid

3.1.15 Difference between throw and throws in Java

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

3.1.16 Built-in exceptions

Exception	Description
ClassNotFoundException	Class requested not found.
CloneNotSupportedException	Occurs when an attempt is made to clone an object of a class that doesn't implement the Cloneable interface.
Exception	Exception and subclasses thereof that an application may want to catch.
IllegalAccessException	Attempted access of class refused.
InstantiationException	Occurs when an attempt is made to instantiate an object from an abstract class or interface.
InterruptedException	Occurs when a thread has been interrupted by another thread.
NoSuchFieldException	Field requested doesn't exist.
NoSuchMethodException	Method requested doesn't exist.
ArithmeticException	Arithmetic exception occurred.
ArrayIndexOutOfBoundsException	Attempt to access array with an illegal index.
ArrayStoreException	Attempt to store incompatible type into object array element.
ClassCastException	Attempt to cast an object to a subclass of which it is not an instance.
IllegalArgumentException	Method invoked with an illegal argument.
IndexOutOfBoundsException	An index is out of bounds.
NullPointerException	Attempt to access an object with a null reference.
NumberFormatException	Invalid attempt to convert the contents of a string to a numeric format..
RuntimeException	Runtimes Exception and subclasses thereof thrown by the *JVM*.

3.2 Input / Output Basics

Java I/O (Input and Output) is used to process the input and produce the output.

Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.

3.2.1 Streams

Stream is basically a channel on which the data flow from sender to receiver. A Stream will be an input stream or, an output stream.

- InputStream − This is used to read data from a source.
- OutputStream − This is used to write data to a destination.



3.2.2 Reading and Writing Streams

3.2.2.1 Reading and Writing Console

In java, System is a class defined in java.lang and in, out and err are the variables of the class System. Hence System.out is an object for standard OutputStream, System.in is an object for standard inputStream and System.err is an object for standard inputStream and error.

print() and println() are the functions used to write the output on the console. These methods belongs to the class PrintWriter.

Method 1: Reading input using Scanner and System.in

Scanner is a class defined in java.util package. With the help of System.in, we can read the console input by creating the instance for the class Scanner.

Example

```
import java.util.*;
class sample{
public static void main(String args[]){
Scanner sc=new Scanner(System.in);
System.out.print("Enter a String: ");
String s=sc.nextLine();
System.out.println("Your String: "+s);
}
}
```

Output

```
Enter a String: Hai
Your String: Hai
```

Method 2: Reading input using BufferedReader, InputStreamReader and System.in

BufferedReader and InputStreamReader are the classes defined in java.in package. With the help of System.in, we can read the console input by creating the instance for the class BufferedReader.

Example: Reading String

```
import java.io.*;
public class sample
{
public static void main(String[] args) throws IOException
{
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
System.out.print("Enter a String: ");
String s = br.readLine();
System.out.println("Your String: "+s);
}
}
```

Output

```
Enter a String: Hai
Your String: Hai
```

Reading Integer

```
import java.io.*;
public class sample {
public static void main(String[] args) throws IOException {
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
System.out. print("Enter a Number: ");
String s = br.readLine();
int n=Integer.parseInt(s);
System.out.println("Your Number: "+n);
}
}
```

Output

```
Enter a Number: 7
Your Number: 7
```

3.2.2.2 Reading and Writing Files

InputStream is an abstract class for streaming the byte input. Various methods defined by this class are as follows:

1. int available()- returns the number of bytes that are available for reading
2. void close()- closes this input stream and releases any system resources associated with the stream.
3. void mark(int readlimit)- marks the current position in this input stream.
4. boolean markSupported()- tests if this input stream supports the mark and reset methods.
5. int read()- reads the next byte of data from the input stream.
6. int read(byte[] b)- reads some number of bytes from the input stream and stores them into the buffer array b.
7. int read(byte[] b, int off, int len)- reads up to len bytes of data starting from the offset off, from the specified byte array b to an array
8. void reset()- repositions this stream to the position at the time the mark method was last called on this input stream.
9. long skip(long n)- skips over and discards n bytes of data from this input stream.

OutputStream is an abstract class for streaming the byte output. Various methods defined by this class are as follows:

1. void close()- closes this output stream and releases any system resources associated with this stream.
2. void flush()- flushes this output stream and forces any buffered output bytes to be written out.
3. void write(byte[] b)- writes b.length bytes from the specified byte array to this output stream.
4. void write(byte[] b, int off, int len)- writes len bytes from the specified byte array starting at offset off to this output stream.
5. void write(int b)- writes the specified byte to this output stream.

FileInputStream class creates an InputStream using which we can read bytes from a file.

Example:

```
import java.io.FileInputStream;
public class DataStreamExample {
public static void main(String args[]){
try{
FileInputStream fin=new FileInputStream("Sample1.txt");
int n=fin.available();
for(int i=0;i<n;i++)
System.out.print((char)fin.read());
fin.close();
}catch(Exception e){System.out.println(e);}
}
}
```

Sample1.txt

```
India is my Country
I love my Country
```

Output

```
India is my Country
I love my Country
```

FileOutputStream class creates an OutputStream using which we can write bytes to a file.

Example:

```
import java.io. FileOutputStream;
public class DataStreamExample {
public static void main(String args[]){
try{
```

```
String s="India is my Country\n"+"I love my Country";
byte n[]=s.getBytes();
FileOutputStream fin=new FileOutputStream("Sample2.txt");
for(int i=0;i<n.length;i++)
fin.write(n[i]);
System.out.println("Data is written to the file...");
fin.close();
}catch(Exception e){System.out.println(e);}
}
}
```

Output

Data is written to the file...

Sample2.txt

India is my Country
I love my Country

3.2.3 Byte Streams and Character Streams

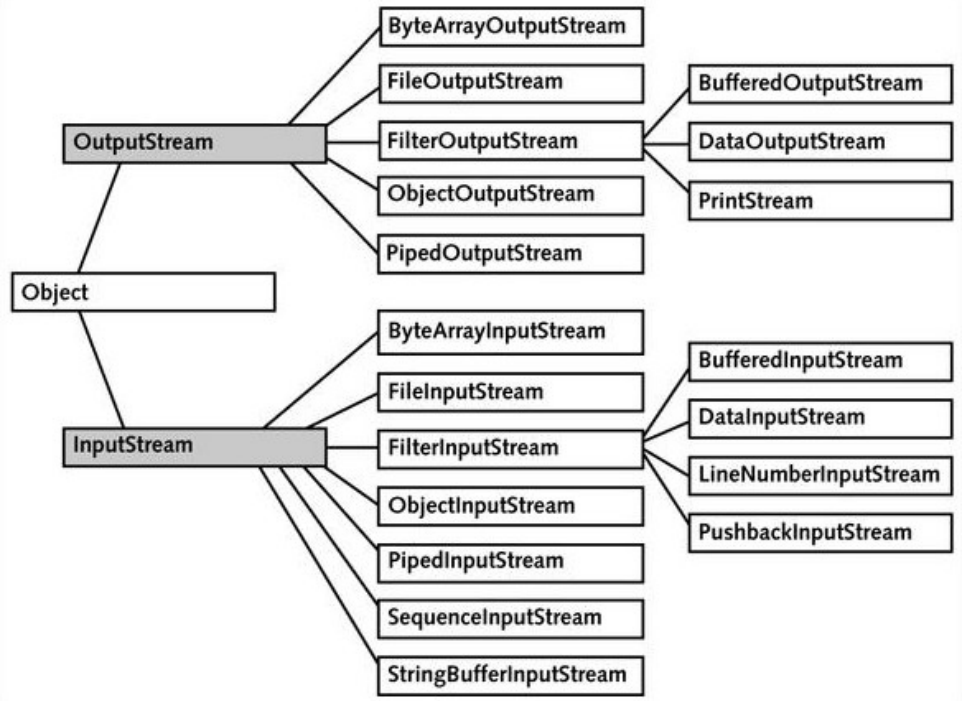
Based on the data they handle there are two types of streams

1. Byte Stream : It provides a convenient means for handling input and output of byte.
2. Character Stream : It provides a convenient means for handling input and output of characters.

Byte Stream

These handle data in bytes (8 bits) i.e., the byte stream classes read/write data of 8 bits. Using these you can store characters, videos, audios, images etc.

At the top of the byte stream class hierarchy, there are two abstract classes: *InputStream* for byte-oriented input and *OutputStream* for byte-oriented output operations. The hierarchical layout is as follows:



Description of the byte-oriented stream classes:

1. **OutputStream** and **InputStream** are abstract
2. **ByteArrayOutputStream** objects are used to create a buffer in memory. All the data that you send to the stream is placed in this buffer.
3. **FileOutputStream** objects are used for sending data to a file.
4. **ObjectOutputStream** objects are used for serializing (saving) objects to various streams (See the serialization topic).
5. **PipedOutputStream** objects are used to implement the write portion of the "piping" concept (We will defer the discussion of this stream to the multithreading topics).

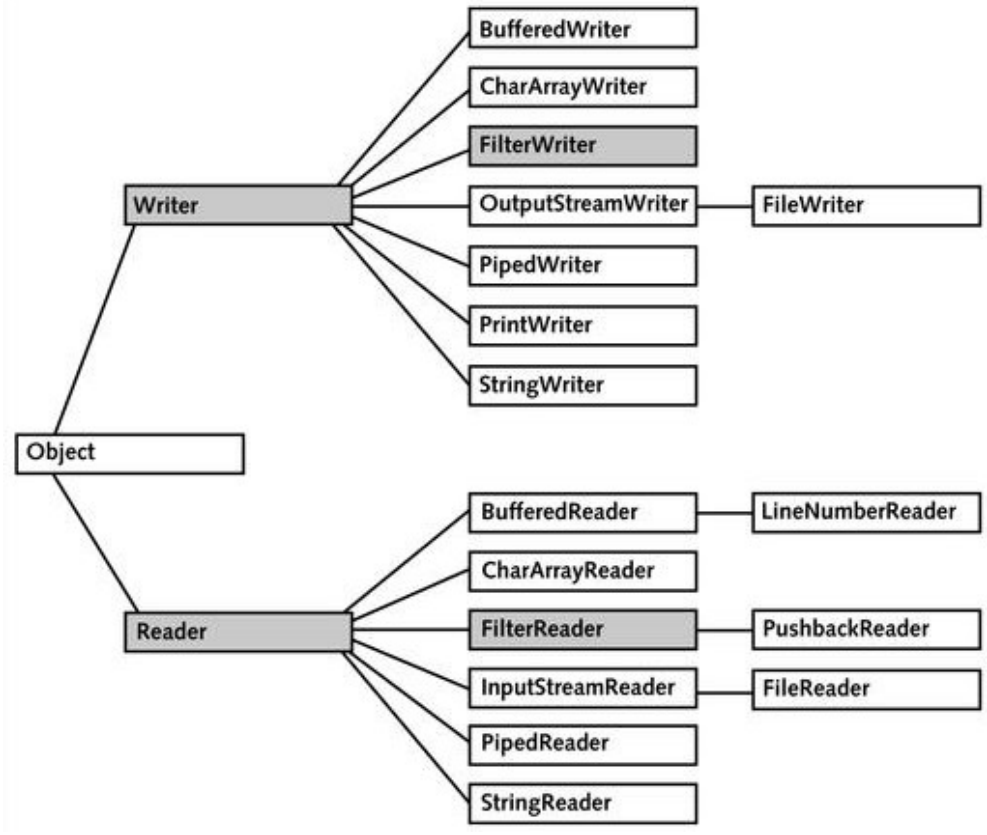
6. **FilterOutputStream** objects are used to process stream output so that it can be buffered and written as blocks of useful data instead of as one byte at a time.
7. **BufferedOutputStream** is a FilterOutputStream class used to buffer data in a stream before writing it. flush() is used to write the data to the stream.
8. **DataOutputStream** is a FilterOutputStream class used to write primitives (int, char, long, etc) to a stream.
9. **PrintStream** is a FilterOutputStream class used for producing formatted output. (DataOutputStream handles the storage of data, PrintStream handles the display of data.)
10. **ByteArrayInputStream** objects allow a buffer in memory to be used as an InputStream.
11. **FileInputStream** objects are used for reading data from a file.
12. **ObjectInputStream** objects are used for deserializing (reading) objects from various streams (See the serialization topic).
13. **PipedInputStream** objects are used to implement the read portion of the "piping" concept (We will defer the discussion of this stream to the multithreading topics).
14. **SequenceInputStream** are used to convert two or more InputStream objects into a single InputStream object.
15. **StringBufferInputStream** objects are used to convert a String object into an InputSteam.
16. **FilterInputStream** objects are used to process stream input so that it can be read as blocks of useful data instead of as one byte at a time.
17. **BufferedInputStream** is a FilterInputStream class used to buffer data in a stream before reading it.
18. **DataInputStream** is a FilterInputStream class used to read primitives (int, char, long, etc) from a stream.
19. **LineNumberInputStream** is a FilterInputStream class used to keep track of line numbers in the input stream (getLineNumber(); setLineNumber(int)).
20. **PushbackInputStream** is a FilterInputStream class that has a one byte push-back buffer so you can push back the last character read.

These classes define several key methods. Two most important are

- read() : reads byte of data.
- write() : Writes byte of data.

The Character Stream Classes

At the top of the character stream class hierarchy, there are two abstract classes: *Reader* for character-oriented input and *Writer* for character-oriented output operations. The hierarchical layout is as follows:



Description of the character-oriented stream classes:

1. **Reader:** Top-level abstract class to read to character streams.
2. **BufferedReader:** Provides an in-between buffer for efficiency while reading text from character input stream.
3. **LineNumberReader:** Uses a buffered character input stream that keeps track of line numbers.
4. **CharArrayReader:** Implements an auto-increasing character buffer that may be used as a reader.
5. **FilterReader:** An instance of this class is used for reading character files.
6. **PushbackReader:** This enables a character to be pushed back into the stream after reading.
7. **InputStreamReader:** An instance of this class provides a bridge from byte streams to character streams. Bytes are decoded into characters using a specified character set.
8. **FileReader:** An instance of this class is used for reading character files.
9. **PipedReader:** Uses a pipe for character input stream.
10. **StringReader:** Character output stream reader from source string.
11. **Writer:** Top-level abstract class to write to character streams.
12. **BufferedWriter:** Provides an in-between buffer for efficiency while writing text to a character output stream.
13. **CharArrayWriter:** Implements an auto-increasing character buffer that may be used as a writer.
14. **FilterWriter:** Abstract class for writing filtered character streams.
15. **OutputStreamWriter:** An instance of this class provides a bridge between character streams and byte streams. Characters are encoded into bytes using a specified character set.
16. **FileWriter:** An instance of this class is used for writing character files.
17. **PipedWriter:** Uses a pipe for character output stream.
18. **PrintWriter:** Prints a formatted representation of an object to a test-output stream.
19. **StringWriter:** Character output stream is collected in a string buffer and may be used for constructing a string.

These classes define several key methods. Two most important are

- read() : reads character of data.
- write() : Writes character of data.

Difference between Byte Stream and Character Stream

Byte Stream	Character Stream
A **byte** stream access the file byte by byte.	A character stream will read a file character by character.
Super Classes 1. InputStream 2. OutputStream	Super Classes 1. Reader 2. Writer
Byte oriented streams do not use any **encoding scheme** while Character oriented streams use character encoding scheme(UNICODE).	Character stream can support all types of character sets ASCII, Unicode, UTF-8, UTF-16 etc.
All byte stream classes are descended from **InputStream** and **OutputStream** .	All character stream classes are descended from Reader and Writer .